Often there are extension-specific config lines following this; let's take the ever-popular xdebug debugging extension as an example… It's quite common to see something like this in an xdebug.ini file:

```
extension=xdebug.so
xdebug.mode=develop,debug,trace
xdebug.discover_client_host=true
xdebug.discover_client_discovery_header=HTTP_X_FORWARDED_FOR
```

The first line has to be the instruction to link the .so file (otherwise none of the xdebug functionality will be available to the PHP binary); the remaining xdebug.* lines are various configuration options. In this case we're telling xdebug to support three different debugging modes, and it should attempt to connect back to the host initiating the connection to start a debugging session. Debugging PHP features to a small extent later in the book, but we'll just be using core PHP functions to do it. I highly recommend XDebug for large projects; you can get started by visiting https://xdebug.org/

Structure and Syntax

Now that we've covered how to install, configure, and run PHP, it's time to start looking at the language itself. You've already seen a few simple snippets of PHP code, which we've used to verify and test the various installation options. Now it's time to see how to build something more complex.

Source Code Files

Previous sections of this chapter have mostly relied on passing PHP code directly to the CLI interpreter, either using php -r (code is entered as part of command line invocation) or php -a (code can be entered via an interactive shell program). For anything more trivial than a few lines, you are best off storing the code in a file; the section on web-server SAPIs demonstrated this.

All PHP code in a file must follow an opening <?php tag, and there must be a closing ?> tag if the file also contains non-PHP code. This has been the case since PHP's earliest versions, when its only purpose was to run as a web-server module. The use of opening tags is required even if the file contains nothing but PHP, although closing tags in this